

1. Virtualizzazione

La virtualizzazione che è il tema che porta più vicino al cloud. Mentre da un lato qualcuno utilizza il silicio per andare veloce, dall'altro lato qualcuno utilizza silicio su cui sia possibile fare la virtualizzazione (chi offre il cloud).

Il processore crea delle zone trusted e deve avere delle caratteristiche architetturali per poter operare verso la virtualizzazione delle risorse. Se un processore non ha quelle caratteristiche non si può fare la virtualizzazione.

Se metti il cloud hai bisogno di virtualizzazione. Vogliamo capire questo processo di virtualizzazione su processori ad esempio della famiglia intel.

Nei sistemi pipelined ogni fase viene controllata da una sua rete di controllo e poi ce n'è una generale.

La virtualizzazione consiste nell'astrazione di risorse, sia di tipo hardware che software, tale per cui l'astrazione esposta è del tutto equivalente alla risorsa fisica sottostante.

Esistono vari esempi di virtualizzazione:

- MMU per la gestione della memoria virtuale;
- SO per la gestione delle risorse fisiche (CPU, memoria e I/O) usate dalle applicazioni;
- Controller dei dischi

La virtualizzazione si basa sulle tecniche di:

- Multiplexing (spaziale o temporale): espone una risorsa fra più entità virtuali;
- Aggregazione: genera una astrazione di una risorsa a partire da più risorse fisiche. Un esempio sono i RAID o lo storage distribuito, dove più dischi fisici separati vengono visti dal sistema come un unico grande volume logico;
- Emulazione: utilizza software per emulare una risorsa virtuale diversa da quella fisica sottostante (es. Bytecode)

Una macchina virtuale è un ambiente di elaborazione completo con capacità di elaborazione, memoria e canali di comunicazione isolati. È un'istanza software di un sistema operativo che gira su un hypervisor.

Ci sono diversi tipi di VM:

- Macchine virtuali basate sul linguaggio come la JVM
- Macchine virtuali leggere: si basano su una combinazione di meccanismi di isolamento hardware e software per garantire che le applicazioni in esecuzione sul processore siano isolate da altre applicazioni e dal sistema operativo sottostante. (container)
- Macchine virtuali a livello di sistema: l'ambiente di elaborazione isolato assomiglia all'hardware di un computer. La VM può eseguire un sistema operativo standard e le sue applicazioni in isolamento rispetto alle altre virtual machine e dal resto dell'ambiente. Per isolamento intendiamo che non vede le altre vm così come le altre vm non vedono lei. Ogni VM ha la copia dell'hardware sottostante ed esegue la propria istanza del SO chiamato sistema operativo guest.

Oggi con il termine macchine virtuali facciamo quasi sempre riferimento alle vm di sistema.

Ogni VM ha il proprio kernel, i propri processi e un ciclo di vita autonomo, come se fosse una macchina reale, ma sfruttando una piattaforma di virtualizzazione.

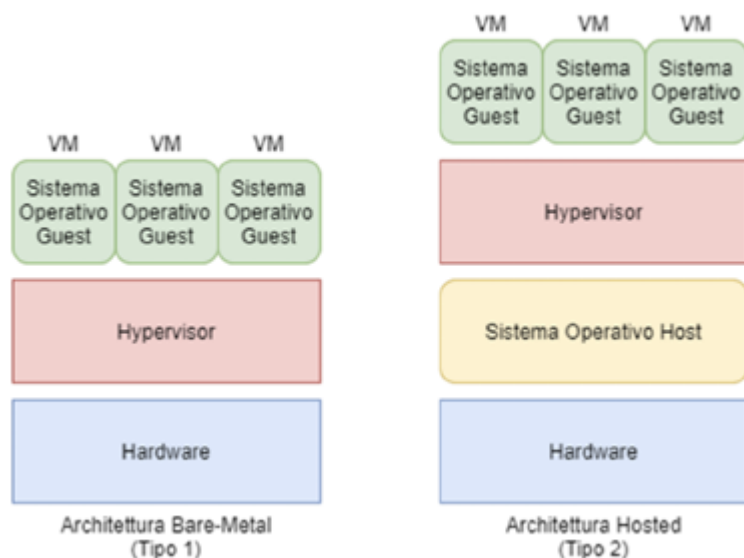
Hypervisor è un software di sistema che gestisce le macchine virtuali per poter gestire l'isolamento. Il SO non vede la macchina ma l'hypervisor. Non può vedere la macchina perché senno non ci sarebbe più la capacità di isolamento. Abbiamo sicuramente aggiunto inefficienza.

Con il 68k lo posso fare solo in un modo ovvero se quando scrivo l'applicazione lo voglio fare perché non ho delle primitive che mi permettono di farlo.

Un hypervisor applica il principio di stratificazione al calcolatore secondo tre criteri:

- **Equivalenza:** la risorsa esposta (ovvero la macchina virtuale) è equivalente al computer sottostante. Questo è un requisito forte, che è rilassato in una certa misura quando l'architettura lo richiede. Il principio di equivalenza stabilisce che qualsiasi programma eseguito all'interno di una macchina virtuale deve mostrare un comportamento identico a quello che manifesterebbe se venisse eseguito direttamente sull'hardware nativo.
- **Sicurezza:** l'isolamento richiede che le macchine virtuali siano isolate l'una dall'altra e dall'hypervisor, il che rafforza la modularità del sistema.
- **Prestazioni:** il sistema virtuale deve mostrare nel peggiore dei casi una lieve diminuzione della velocità e quindi significa che deve essere efficiente. Questo requisito distingue gli hypervisor dai simulatori di macchine, che invece rallentano significativamente l'esecuzione del sistema target.

Esistono due tipi di hypervisor:



Un Hypervisor di tipo 1 (bare-metal) controlla direttamente tutte le risorse del sistema host e cioè si trova direttamente sull'hardware (utilizzo nei data center), mentre un hypervisor di tipo 2 (Hosted) opera su un sistema operativo host efficiente.

Nel tipo 2 l'hypervisor vive al di sopra del sistema operativo. Più livelli mettiamo, più è inefficiente il sistema. VMware, Xen e Microsoft Hyper-V sono di tipo 1,

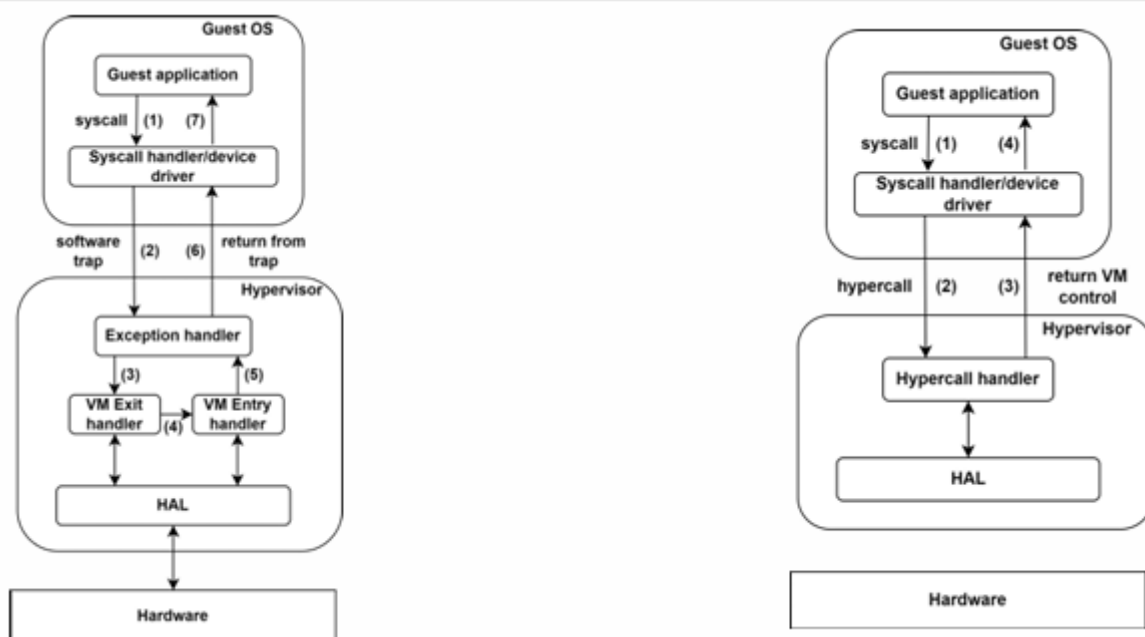
Mentre VMware Workstation, VMware Fusion, KVM, Microsoft VirtualPC, Parallels e Oracle VirtualBox sono tutti hypervisor di tipo 2

Su una macchina bare l'hypervisor deve eseguire lo scheduling del sistema e l'allocazione (reale) delle risorse mentre, nel caso di esecuzione su un SO, l'hypervisor deve solo occuparsi di creare le macchine virtuali.

Un Hypervisor è anche responsabile di garantire la sicurezza della macchina virtuale assicurando che la CPU virtuale venga eseguita con privilegi ridotti. L'esecuzione diretta della macchina virtuale porta a frequenti trap ogni volta che il SO guest tenta di eseguire un'istruzione privilegiata, che deve essere emulata dall'hypervisor (TRAP AND EMULATE)

La **Virtualizzazione Completa (Trap-and-Emulate)** garantisce sicurezza e isolamento declassando il Sistema Operativo guest in un anello a privilegi ridotti (es. Ring 1), lasciando all'Hypervisor il controllo esclusivo dell'hardware (Ring 0). Per mantenere alte le prestazioni, le istruzioni standard vengono eseguite nativamente dalla CPU fisica (*Direct Execution*). Quando però il Guest OS tenta di eseguire un'operazione critica o privilegiata, l'hardware la blocca istantaneamente generando un'eccezione (*Trap*); a questo punto l'Hypervisor interviene, emula l'istruzione al posto della macchina virtuale e le restituisce l'esecuzione. Pur garantendo un isolamento perfetto, i continui cambi di contesto (*context switch*) tra Guest e Hypervisor rallentano il sistema, creando colli di bottiglia che oggi vengono superati grazie a specifiche estensioni hardware (come Intel VT-x).

1.2 Tipi di Virtualizzazione



- **Full-Virtualization:** Il Sistema Operativo guest **non è modificato**. Crede di essere installato su un computer fisico vero e proprio e non sa di essere virtualizzato.
 - **Il Meccanismo:** Il SO prova a eseguire un'istruzione privilegiata direttamente sull'hardware (perché crede di avere i permessi massimi, il Ring 0). L'hardware lo blocca istantaneamente generando un'**eccezione hardware (Trap)**.
 - **L'intervento:** È l'Hypervisor che si accorge del blocco, cattura l'eccezione, emula l'operazione di nascosto per conto del SO e gli restituisce il controllo.
- **Para-virtualization:** Il Sistema Operativo guest **viene riscritto e modificato** (ecco perché usi driver specifici o kernel adattati). Sa perfettamente di trovarsi all'interno di una macchina virtuale.

- **Il Meccanismo (Hypercall):** Il SO non prova nemmeno a toccare direttamente l'hardware, perché sa che gli è vietato. Al suo posto, effettua una chiamata diretta all'Hypervisor per chiedergli di compiere quell'azione.
- **Trap Software vs System Call:** Un'applicazione utente fa una *System Call* al SO; il SO (modificato) fa una *Hypercall* (tramite trap software, es. l'interruzione programmata `int 0x82` in Xen) all'Hypervisor.

buco di virtualizzazione dell'architettura x86

Perché il meccanismo di *Trap-and-Emulate* funzioni perfettamente, il processore deve rispettare una regola matematica: **tutte le istruzioni "sensibili"** (quelle che leggono o modificano lo stato globale del sistema) **devono essere anche "privilegiate"** (cioè devono generare obbligatoriamente un Trap se eseguite fuori dal Ring 0).

I processori Intel x86 classici (quelli prima del 2005) non erano stati progettati per essere virtualizzati. Contenevano **17 istruzioni "ribelli"** (come `POPF`, `SGDT`, `SIDT`) che erano sensibili, ma non privilegiate. Cosa significava all'atto pratico?

- Se il Sistema Operativo guest provava a eseguire una di queste 17 istruzioni, **la CPU non generava nessuna eccezione (nessun Trap)**.
- L'istruzione veniva eseguita in silenzio, ma falliva o restituiva dati
- L'Hypervisor rimaneva completamente **"cieco"**: non essendoci stato alcun Trap, non sapeva che il Guest OS aveva provato a fare un'operazione critica, non poteva intervenire per emularla, e il Sistema Operativo semplicemente andava in crash o impazziva.

Ecco quindi che con la Para-virtualizzazione:

- Hanno preso il codice sorgente di Linux (Guest OS).
- Hanno cercato tutte le occorrenze di quelle 17 istruzioni problematiche.
- Le hanno cancellate e sostituite con delle **Hypercall** esplicite e sicure.

Istruzione Sensibile: Un'istruzione si definisce **sensibile** se altera o rivela lo stato globale della macchina. Riguarda il *cosa* l'istruzione cerca di fare. Si divide solitamente in due categorie:

- **Sensibile al controllo (Control-sensitive):** Tenta di modificare la configurazione hardware, l'allocazione delle risorse o il livello di privilegio. Ad esempio: modificare i registri della MMU (Memory Management Unit) per alterare la paginazione, o **disabilitare gli interrupt** a livello globale.
- **Sensibile al comportamento (Behavior-sensitive):** Il suo risultato dipende dallo stato reale dell'hardware, oppure rivela informazioni sulla macchina fisica. Ad esempio: leggere l'orologio di sistema reale o leggere l'indirizzo fisico di base delle tabelle dei descrittori

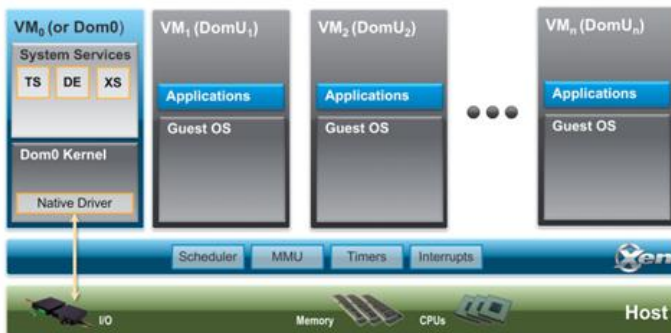
Se un Sistema Operativo guest esegue un'istruzione sensibile, l'Hypervisor **deve** poterla intercettare, altrimenti il guest prenderà il controllo del silicio o scoprirà di essere in una simulazione.

Istruzione Privilegiata (La protezione hardware)

Un'istruzione si definisce **privilegiata** se il processore è fisicamente cablato per bloccarla tramite un'eccezione hardware (un **Trap**) qualora venga eseguita al di fuori dell'anello di massimo privilegio (Ring 0 / Kernel Mode). Riguarda il come la CPU si difende. Le trap devono essere scatenate quando delle istruzioni sensitive vengono eseguite con livelli di privilegi utente.

Xen

1.3 XEN



XEN è un hypervisor di tipo 1 per cui esegue a diretto contatto con l'hardware sottostante.

Nonostante sia nato per eseguire solo tramite tecnica di paravirtualization, versioni successive permettono l'esecuzione anche in modalità full virtualization.

Xen offre primitive per la full-virtualization tramite la modalità HVM (Hardware Virtual Machine).

Le primitive fanno uso di un set di API specifiche per l'architettura di riferimento. Nel caso di architetture Intel x86, ci sono primitive per la gestione di VMExit e VMEntry, per l'accesso alla struttura di controllo VMCS, ecc.

Le primitive per la full-virtualization si appoggiano a un set di primitive general-purpose a disposizione di Xen:

- Gestione della memoria
- Schedulazione di macchine virtuali
- Allocazione di risorse fisiche

1.4 Cos'è la virtualizzazione

La virtualizzazione astrae i dettagli di implementazione di una risorsa. La virtualizzazione riguarda:

- File manager per accedere allo storage
- Primitive I/O per comunicare con le periferiche
- Memoria virtuale per l'allocazione dinamica dei processi alla memoria fisica.

La virtualizzazione viene estesa alle risorse fisiche:

- Virtualizzazione della CPU, memoria fisica, periferiche I/O

1.5 Virtualizzazione della cpu

È possibile solo quando viene soddisfatto un requisito specifico: tutte le istruzioni sensibili possono essere emulate. Le istruzioni sensibili sono quelle che possono essere eseguite solo con i privilegi a livelli di kernel. Ci sono diverse tecniche per emulare le istruzioni sensibili:

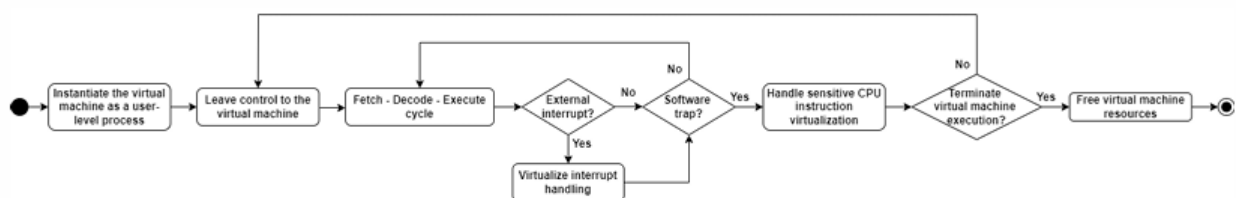
- DBT
- Para virtualizzazione

- Full- virtualization

DBT sostituisce i TRAP con i salti a subroutine gestite dall'hypervisor.

Le sensitive instruction sono istruzioni che posso eseguire in diversi stati. Quando le eseguo si comportano in modo differente in base allo stato in cui mi trovo. Queste istruzioni non tutte sono privilegiate. Perché questo sistema va male quando voglio fare una macchina virtuale? perché quando faccio una macchina virtuale quella azione la voglio voler fare. Se setto dei privilegi io non posso farlo perché interagisce con gli altri sistemi operativi. Le istruzioni sensitive credo di averla fatta, ma in realtà non l'ho fatta. Se invece se sto in un sistema in cui ho operazioni sensitive e privilegi, quello che non posso fare ma non dovrei farlo attiva l'hypervisor.

Sistemi che hanno più livelli di interruzioni sono interessanti perché consentono di fare controlli successivi. Più scendo e più posso fare operazioni che hanno maggior vantaggio, ma sto sempre nello stesso sistema operativo, finché non arrivo al nucleo.



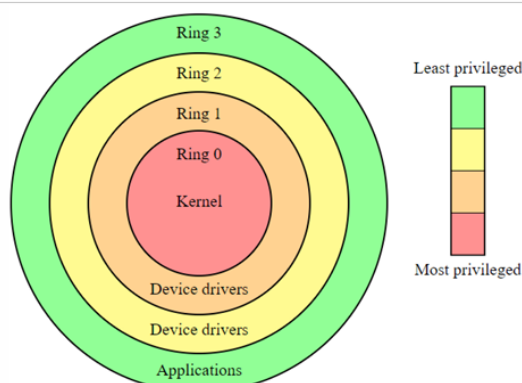
1.6 Virtualizzazione in intel x86

La cpu intel definisce 4 livelli di privilegi chiamati RING. Il primo ring (ring0) è il più alto mentre il ring 3 è il più basso.

L'esecuzione delle istruzioni deve avvenire con i privilegi appropriati:

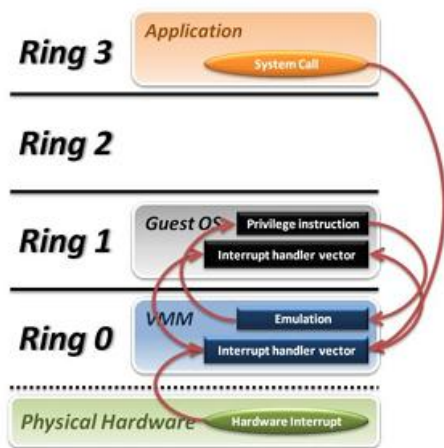
- Il codice utente dovrebbe essere eseguito all'interno del ring3
- Il codice del kernel dovrebbe essere eseguito in anelli con privilegi più elevati.
 - o ISR code
 - o System Call

Intel CPUs execution privileges



Le istruzioni eseguite senza privilegi previsti potrebbe portare a un comportamento diverso.

Il sistema operativo guest e le applicazioni si trovano sui ring da 1 a 3. Tutte le system call e le interrupt hardware sono gestite dall'hypervisor che si trova sul ring 0.



Tutte le istruzioni sensibili sono virtualizzate dal DBT (dynamic binary translation).

Il problema storico: L'architettura x86 classica non era stata progettata per essere virtualizzata

Visto che non potevano affidarsi all'hardware per generare i Trap, l'Hypervisor doveva letteralmente "leggere" e tradurre in tempo reale (dinamicamente) il codice binario del Guest OS. Quando incontrava un'istruzione problematica, la rimpiazzava al volo con una sequenza di istruzioni sicure che richiama l'Hypervisor.

L'ISA originale non era stata pensata per il trap-and-emulate. Come detto in precedenza alcune istruzioni sensibili, se eseguite fuori dal Ring 0, non facevano scattare nessun allarme hardware rendendo cieco l'hypervisor

1.7 Virtualizzazione Technology support (VT-x)

Come detto il problema era risolvibile con la Para-virtualization, ma era abbastanza complicato e quindi l'Intel ha sviluppato la VT-x che è l'estensione hardware che consente di supportare la virtualizzazione nelle architetture x86-64.

La vera genialata di Intel non è stata aggiungere altri "anelli", ma creare due mondi paralleli, chiamati **Operation Modes**. Questa estensione prevede due operazioni: VMX root and non-root operation.

- **VMX Root Operation:** Il regno dell'Hypervisor. Qui si ha il controllo totale, superiore persino a quello di un normale Sistema Operativo.
- **VMX Non-Root Operation:** Il recinto dove vengono fatte girare le Macchine Virtuali (i Guest OS).

Ogni mode operation ha i 4 livelli di privilegi. Lo switch tra i due mode operation avviene quando le istruzioni sensibili sono eseguite durante le non-root operation.

Quando il Guest OS viene messo in modalità *Non-Root*, ha a disposizione i **suoi** 4 anelli (Ring 0-3). Quindi il kernel del Guest gira felicemente nel suo Ring 0, convinto di essere il padrone assoluto dell'hardware.

1.8 Virtual machine control structure (VMCS)

L'hardware mantiene due tipi di informazioni di stato: **guest-area e host-area**

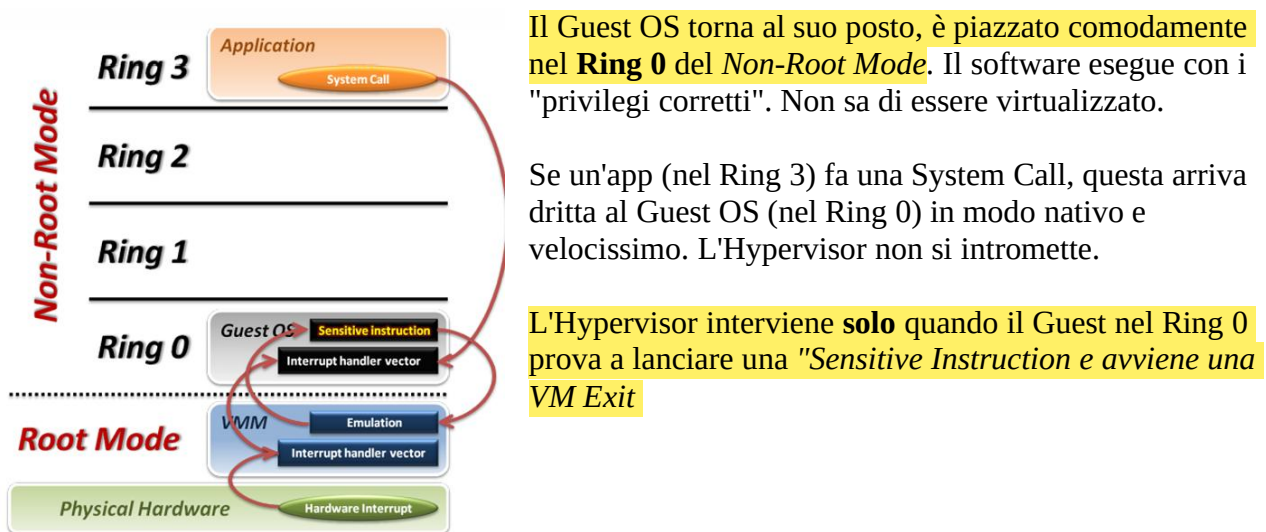
Le applicazioni che girano nell'area host gestendo l'esecuzione delle macchine virtuali utilizzano la VMCS. Viene usata dall'hypervisor durante la gestione dei cambi di contesto da Vm a host e anche da host a VM.

Questi cambi di contesto si chiamano VM Exit e VM Entry.

Questa struttura contiene informazioni sullo stato sia della VM che della macchina host. Viene caricata quando la macchina virtuale esce per gestire l'emulazione delle istruzioni e viene caricata anche quando la VM riprende il controllo per ripristinare lo stato precedente.

Quando il Guest OS, dal suo finto Ring 0 in modalità Non-Root, prova a eseguire un'istruzione critica (es. toccare un registro fisico reale o l'hardware I/O), l'hardware della CPU lo intercetta istantaneamente. A questo punto si innesca l'**Operation mode switch** (tecnicamente chiamato **VM Exit**): la CPU congela il mondo *Non-Root*, salva lo stato nella guest area della VMCS, e passa l'esecuzione al mondo *Root* (l'Hypervisor) che gestisce la situazione, emula l'istruzione, andando a leggere sempre nella VMCS il motivo per cui c'è stato il VM Exit e poi fa ripartire il Guest con un **VM Entry**.

Le virtual machine operano in VMX non-root mode



Perché non modificare l'unica modalità esistente?

In linea teorica, Intel avrebbe potuto mantenere un'unica modalità d'esecuzione e ridefinire quelle 17 istruzioni come "privilegiate", forzando una trap ogni volta che venivano eseguite al di fuori del Ring 0. Tuttavia, questa strada era impraticabile per due ragioni ingegneristiche fondamentali:

Molti programmi e giochi legittimi in User Mode (Ring 3) usavano storicamente alcune di queste istruzioni (es. `SGDT` per scopi di debug). Se Intel le avesse trasformate improvvisamente in istruzioni privilegiate su scala globale, milioni di software esistenti nel mondo avrebbero iniziato a crashare con un errore di protezione hardware (*General Protection Fault*). Preservare la retrocompatibilità totale era un vincolo commerciale imprescindibile.

1.9 Virtualizzazione degli interrupt esterni

Qualsiasi interrupt esterno provoca un VM Exit.

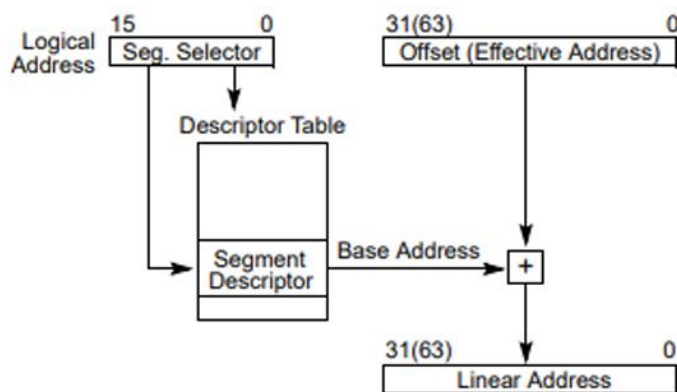
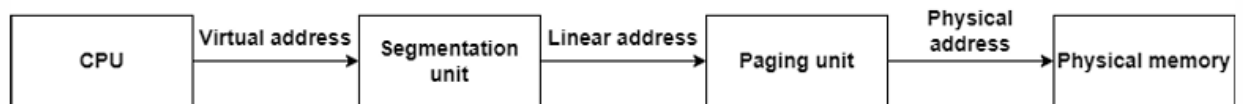
L'Hypervisor blocca l'evento, guarda se il Guest può riceverlo, controllando il flag interrupt in un registro. se l'interupt flag è pari a 1 allora la VM può ricevere interrupt e quindi l'hypervisor inietta l'evento nella VMCS ed esegue VM Entry.

Se il flag è =0 non possiamo ne consegnare l'interrupt e ne scartarlo, allora si attende che il flag torni uguale a 1 per poter passare l'interrupt.

1.10 virtualizzazione della memoria in intel x86x64

la virtualizzazione nelle architetture bare-metal segue due step:

- l'indirizzamento lineare
- Paginazione



Nella fase di segmentazione l'indirizzo logico viene diviso in segmento e offset. Il segmento viene usato come indice nella tabella per trovare il base address. Poi sommiamo il base address all'offset

Successivamente l'indirizzo entra nella Paging Unit: l'hardware suddivide la memoria in "Pagine" fisse da 4 KByte. Per trovare la pagina giusta in mezzo a gigabyte di RAM, usa un sistema ad albero gerarchico a 4 livelli, come se dovesse cercare Nazione -> Regione -> Provincia -> Comune.

Il registro **CR3** (gestito dal Sistema Operativo) contiene l'indirizzo fisico di partenza della primissima tabella (PML4).

L'hardware prende i primi 9 bit dell'indirizzo lineare, va nella tabella PML4 e trova il puntatore alla tabella successiva (PDPTE).

E così anche per gli altri campi fino ad arrivare alla PAGE TABLE (PTE). Quest'ultima contiene l'indirizzo base fisico della pagina in RAM.

In un ambiente virtualizzato, il sistema operativo "guest" (quello dentro la VM) crede di avere il controllo totale sulla RAM fisica. In realtà, sta gestendo una memoria fisica fittizia (Guest Physical Address), che deve essere a sua volta mappata sulla vera RAM della macchina fisica "host".

Una volta ottenuto il Guest Physical Address, entra in gioco la scatola EPT translation mechanism.

In passato, questa traduzione veniva fatta via software dall'hypervisor tramite le Shadow Page Tables, un processo lento. L'EPT sposta questo lavoro nell'hardware. Prende il Guest Physical Address e lo traduce nel vero Physical Address (l'indirizzo host reale).